SQL Assignmnet on TechMart

This word consist of code + result (in each ans section) + action output (in last page after all answers solved)

---

1. Display all products that cost more than $500 and have at least 50 items in stock, sorted by price from highest to lowest.

```sql
-- 1. Products > $500 and Stock ≥ 50
SELECT product_id, product_name, price, stock_quantity
FROM Products
WHERE price > 500 AND stock_quantity >= 50
ORDER BY price DESC;
```

Result Grid | Filter Rows: | Edit: | Export/Imp

product_id	product_name	price	stock_quantity
101	UltraBook Pro	1299.99	50
102	SmartPhone X	899.99	120
115	Smartphone Y	599.99	65
NULL	NULL	NULL	NULL

2. Find all customers who have registered in 2023 and have more than 200 loyalty points. Show their full name as a single column, email, and city.

```sql
-- 2. Customers registered in 2023 with >200 loyalty points
SELECT CONCAT(first_name, ' ', last_name) AS full_name, email, city
FROM Customers
WHERE YEAR(registration_date) = 2023 AND loyalty_points > 200;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

full_name	email	city
John Smith	john.smith@email.com	Austin
Michael Williams	michael.w@email.com	Chicago
Robert Jones	robert.j@email.com	Los Angeles
Ava Davis	ava.d@email.com	Portland
Isabella Martinez	isabella.m@email.com	Phoenix

Result 2

3. Update the loyalty points for customers in Texas by adding 50 points to their current total.

```sql
-- Perform the update
UPDATE Customers
SET loyalty_points = loyalty_points + 50
WHERE state = 'TX';

-- View the result
SELECT customer_id, first_name, last_name, state, loyalty_points
FROM Customers
WHERE state = 'TX';
```

Result Grid | Filter Rows: | Edit: | Export/Import:

customer_id	first_name	last_name	state	loyalty_points
1	John	Smith	TX	300
NULL	NULL	NULL	NULL	NULL

4. Create a query to identify products that need restocking (stock_quantity less than 30).

```sql
-- 4. Products that need restocking (stock < 30)
SELECT product_id, product_name, stock_quantity
FROM Products
WHERE stock_quantity < 30;
```

Result Grid | Filter Rows: | Edit: | Ex

product_id	product_name	stock_quantity
107	Desktop Computer	25
NULL	NULL	NULL

5. Calculate the average price of products in each category, but only include categories where the average price is greater than $200.

```sql
-- 5. Average price per category > $200
SELECT pc.category_name, AVG(p.price) AS avg_price
FROM Products p
JOIN ProductCategories pc ON p.category_id = pc.category_id
GROUP BY pc.category_name
HAVING AVG(p.price) > 200;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category_name	avg_price
Electronics	343.323333
Computers	842.490000
Smartphones	516.656667
Consoles	499.990000

6. Find all customers who haven't placed any orders yet.

```sql
-- 6. Customers with no orders
SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS full_name
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	full_name

7. Display the top 5 customers who have spent the most money on purchases, showing their name and total amount spent.

```sql
-- 7. Top 5 customers by total spending
SELECT CONCAT(c.first_name, ' ', c.last_name) AS full_name, SUM(o.total_amount) AS total_spent
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id
ORDER BY total_spent DESC
LIMIT 5;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

full_name	total_spent
John Smith	2599.96
Emma Johnson	2029.97
William Miller	1599.99
Michael Williams	1229.96
Isabella Martinez	1099.98

8. For each product, show its name, price, and the average rating it has received.

```sql
-- 8. Product name, price, average rating
SELECT p.product_name, p.price, ROUND(AVG(r.rating), 2) AS avg_rating
FROM Products p
LEFT JOIN Reviews r ON p.product_id = r.product_id
GROUP BY p.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	price	avg_rating
UltraBook Pro	1299.99	4.50
SmartPhone X	899.99	4.50
Noise Cancelling Headphones	249.99	5.00
Gaming Console Pro	499.99	3.50
Smart Watch	199.99	3.50
Bluetooth Speaker	129.99	5.00
Desktop Computer	1599.99	4.00
Tablet Pro	699.99	5.00
Wireless Gaming Mouse	89.99	4.00
Coffee Maker	149.99	5.00
Microwave Oven	179.99	3.00
Gaming Keyboard	129.99	5.00
Wireless Earbuds	159.99	4.00
External Hard Drive	119.99	NULL
Smartphone Y	599.99	NULL
Ultra HD Monitor	349.99	5.00
Smart Blender	89.99	NULL
Gaming Headset	149.99	NULL
Wireless Router	129.99	NULL
Portable Power Bank	49.99	NULL

9. Find all products that were ordered in February 2024, along with the customers who ordered them.

```sql
-- 9. Products ordered in February 2024 with customer names
SELECT DISTINCT p.product_name, CONCAT(c.first_name, ' ', c.last_name) AS customer_name
FROM Orders o
JOIN OrderItems oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
JOIN Customers c ON o.customer_id = c.customer_id
WHERE o.order_date BETWEEN '2024-02-01' AND '2024-02-29';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	customer_name
Gaming Headset	Ava Davis
Wireless Earbuds	Ava Davis
Portable Power Bank	Ava Davis
Ultra HD Monitor	John Smith

10. List all employees along with their direct manager's name (if they have one).

```sql
-- 10. Employees with their manager's name
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS employee_name,
       CONCAT(m.first_name, ' ', m.last_name) AS manager_name
FROM Employees e
LEFT JOIN Employees m ON e.manager_id = m.employee_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	employee_name	manager_name
501	Thomas Wilson	NULL
502	Sarah Johnson	Thomas Wilson
503	Daniel Brown	Thomas Wilson
504	Jessica Smith	Sarah Johnson
505	Andrew Davis	Thomas Wilson
506	Lisa Miller	Thomas Wilson
507	Robert Taylor	Thomas Wilson
508	Emily Anderson	Jessica Smith
509	Michael Clark	Jessica Smith
510	Amanda Wright	Andrew Davis
511	Joshua Garcia	Andrew Davis
512	Melissa Martinez	Lisa Miller
513	Christopher Lee	Robert Taylor
514	Nicole Harris	Jessica Smith
515	Brian Young	Andrew Davis

11. Find all products that have never been reviewed by any customer.

```sql
-- 11. Products never reviewed
SELECT p.product_id, p.product_name
FROM Products p
LEFT JOIN Reviews r ON p.product_id = r.product_id
WHERE r.review_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Cor

product_id	product_name
114	External Hard Drive
115	Smartphone Y
117	Smart Blender
118	Gaming Headset
119	Wireless Router
120	Portable Power Bank

12. Identify customers who have purchased products from at least 3 different product categories.

```sql
-- 12. Customers who bought from ≥3 categories
SELECT o.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name
FROM Orders o
JOIN OrderItems oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
JOIN Customers c ON o.customer_id = c.customer_id
GROUP BY o.customer_id
HAVING COUNT(DISTINCT p.category_id) >= 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name
1	John Smith
2	Emma Johnson
4	Sophia Brown
5	Robert Jones
6	Olivia Garcia
8	Ava Davis
9	James Rodriguez
10	Isabella Martinez
11	David Anderson
12	Mia Taylor

13. Find all products that are not currently part of any active promotion.

```sql
-- 13. Products not in any active promotion
SELECT p.product_id, p.product_name
FROM Products p
LEFT JOIN ProductPromotions pp ON p.product_id = pp.product_id
LEFT JOIN Promotions pr ON pp.promotion_id = pr.promotion_id AND pr.is_active = TRUE
WHERE pr.promotion_id IS NULL;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name
101	UltraBook Pro
102	SmartPhone X
103	Noise Cancelling Headphones
104	Gaming Console Pro
105	Smart Watch
106	Bluetooth Speaker
107	Desktop Computer
108	Tablet Pro
109	Wireless Gaming Mouse
110	Coffee Maker
111	Microwave Oven
112	Gaming Keyboard
113	Wireless Earbuds
114	External Hard Drive
115	Smartphone Y
116	Ultra HD Monitor
117	Smart Blender
118	Gaming Headset
119	Wireless Router

14. List customers who have purchased the same product more than once across different orders

```sql
-- 14. Customers who bought same product in different orders
SELECT o.customer_id, p.product_name
FROM Orders o
JOIN OrderItems oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
GROUP BY o.customer_id, oi.product_id
HAVING COUNT(DISTINCT o.order_id) > 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	product_name

15. Find departments with more than 3 employees, showing the department name and the number of employees.

```sql
-- 15. Departments with >3 employees
SELECT department, COUNT(*) AS num_employees
FROM Employees
GROUP BY department
HAVING COUNT(*) > 3;
```

Result Grid | Filter Rows: | Export: | Wrap C

department	num_employees
IT	4
Sales	4

16. Calculate the total revenue generated for each month in 2023, sorted chronologically

```sql
-- 16. Monthly revenue for 2023
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS revenue
FROM Orders
WHERE YEAR(order_date) = 2023
GROUP BY month
ORDER BY month;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

month	revenue
2023-05	2449.97
2023-06	879.97
2023-07	1029.97
2023-08	2049.97
2023-09	469.97
2023-10	1099.98
2023-11	959.95
2023-12	1249.96

17. Find the most popular product category based on the number of orders placed.

```sql
-- 17. Most popular category (by orders)
SELECT pc.category_name, COUNT(*) AS total_orders
FROM OrderItems oi
JOIN Products p ON oi.product_id = p.product_id
JOIN ProductCategories pc ON p.category_id = pc.category_id
GROUP BY pc.category_id
ORDER BY total_orders DESC
LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rc

category_name	total_orders
Smartphones	11

18. Identify the employees who earn a salary higher than the average salary of their department.

```sql
-- 18. Employees earning more than department average
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS name, e.department, e.salary
FROM Employees e
JOIN (
    SELECT department, AVG(salary) AS avg_salary
    FROM Employees
    GROUP BY department
) avg_dept ON e.department = avg_dept.department
WHERE e.salary > avg_dept.avg_salary;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	name	department	salary
501	Thomas Wilson	Executive	250000.00
504	Jessica Smith	IT	120000.00
505	Andrew Davis	Sales	110000.00
506	Lisa Miller	Marketing	110000.00
507	Robert Taylor	HR	105000.00

19. Calculate the percentage of reviews that are 4 stars or higher for each product that has at least 2 reviews.

```
-- 19. % of 4+ star reviews for products with ≥2 reviews
SELECT p.product_name,
       COUNT(CASE WHEN r.rating >= 4 THEN 1 END) * 100.0 / COUNT(*) AS percent_high_ratings
FROM Reviews r
JOIN Products p ON r.product_id = p.product_id
GROUP BY r.product_id
HAVING COUNT(*) >= 2;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	percent_high_ratings
UltraBook Pro	100.00000
SmartPhone X	100.00000
Gaming Console Pro	50.00000
Smart Watch	50.00000
Tablet Pro	100.00000
Wireless Earbuds	100.00000

20. Find the average order value for each payment method, and include the number of orders placed with each method.

```
-- 20. Average order value and order count by payment method
SELECT payment_method,
       ROUND(AVG(total_amount), 2) AS avg_order_value,
       COUNT(*) AS order_count
FROM Orders
GROUP BY payment_method;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_method	avg_order_value	order_count
Credit Card	754.27	7
PayPal	625.98	5
Debit Card	372.49	4
Credit Card	764.98	4

21. Create a query to analyze which day of the week has the highest number of orders

```
-- 21. Day of week with most orders
SELECT DAYNAME(order_date) AS day_of_week, COUNT(*) AS order_count
FROM Orders
GROUP BY day_of_week
ORDER BY order_count DESC
LIMIT 1;


-- 22. Quarterly sales trends in 2023 by category
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

day_of_week	order_count
Monday	4

22. Generate a report showing the quarterly sales trends for 2023, broken down by product category

```
-- 22. Quarterly sales trends in 2023 by category
SELECT
    CONCAT(YEAR(o.order_date), '-Q', QUARTER(o.order_date)) AS quarter,
    pc.category_name,
    SUM(oi.unit_price * oi.quantity - oi.discount) AS total_sales
FROM Orders o
JOIN OrderItems oi ON o.order_id = oi.order_id
JOIN Products p ON oi.product_id = p.product_id
JOIN ProductCategories pc ON p.category_id = pc.category_id
WHERE YEAR(o.order_date) = 2023
GROUP BY quarter, pc.category_name
ORDER BY quarter, pc.category_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

quarter	category_name	total_sales
2023-Q2	Audio	629.97
2023-Q2	Computers	1299.99
2023-Q2	Consoles	499.99
2023-Q2	Smartphones	899.99
2023-Q3	Audio	539.97
2023-Q3	Computers	1679.98
2023-Q3	Electronics	899.98
2023-Q3	Kitchen	149.99
2023-Q3	PC Gaming	179.98
2023-Q3	Smartphones	49.99
2023-Q4	Audio	159.99
2023-Q4	Computers	699.98
2023-Q4	Electronics	129.99
2023-Q4	Kitchen	329.98
2023-Q4	PC Gaming	219.98
2023-Q4	Smartphones	1649.94

23. Create a view that displays a summary of each customer's purchase history, including their most purchased category and total spend.

```sql
-- 23. View: Summary of customer purchase history
CREATE VIEW CustomerPurchaseSummary AS
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    SUM(o.total_amount) AS total_spent,
    (
        SELECT pc.category_name
        FROM OrderItems oi2
        JOIN Products p2 ON oi2.product_id = p2.product_id
        JOIN ProductCategories pc ON p2.category_id = pc.category_id
        JOIN Orders o2 ON oi2.order_id = o2.order_id
        WHERE o2.customer_id = c.customer_id
        GROUP BY pc.category_name
        ORDER BY COUNT(*) DESC
        LIMIT 1
    ) AS most_purchased_category
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id;

SELECT * FROM CustomerPurchaseSummary;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	customer_name	total_spent	most_purchased_category
1	John Smith	2599.96	Computers
2	Emma Johnson	2029.97	Smartphones
3	Michael Williams	1229.96	Audio
4	Sophia Brown	779.97	Audio
5	Robert Jones	729.96	Kitchen
6	Olivia Garcia	999.96	PC Gaming
7	William Miller	1599.99	Computers
8	Ava Davis	709.97	Audio
9	James Rodriguez	219.98	PC Gaming
10	Isabella Martinez	1099.98	Smartphones
11	David Anderson	679.97	PC Gaming
12	Mia Taylor	279.98	Audio

24. Calculate the employee hierarchy depth, showing each employee's level in the organization (CEO being level 1).

```sql
-- 24. Employee hierarchy depth (CEO = level 1)
WITH RECURSIVE EmployeeHierarchy AS (
  SELECT employee_id, manager_id, 1 AS level
  FROM Employees
  WHERE manager_id IS NULL
  UNION ALL
  SELECT e.employee_id, e.manager_id, eh.level + 1
  FROM Employees e
  JOIN EmployeeHierarchy eh ON e.manager_id = eh.employee_id
)
SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS name, level
FROM EmployeeHierarchy eh
JOIN Employees e ON e.employee_id = eh.employee_id
ORDER BY level;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

employee_id	name	level
501	Thomas Wilson	1
502	Sarah Johnson	2
503	Daniel Brown	2
505	Andrew Davis	2
506	Lisa Miller	2
507	Robert Taylor	2
504	Jessica Smith	3
510	Amanda Wright	3
511	Joshua Garcia	3
515	Brian Young	3
512	Melissa Martinez	3
513	Christopher Lee	3
508	Emily Anderson	4
509	Michael Clark	4
514	Nicole Harris	4

25. Analyze the correlation between product price and customer review ratings. Do more expensive products tend to get higher ratings?

```sql
-- 25. Correlation: Product price vs. average rating
SELECT
    p.product_name,
    p.price,
    ROUND(AVG(r.rating), 2) AS avg_rating
FROM Products p
JOIN Reviews r ON p.product_id = r.product_id
GROUP BY p.product_id
HAVING COUNT(r.review_id) >= 1
ORDER BY p.price;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Conte

product_name	price	avg_rating
Wireless Gaming Mouse	89.99	4.00
Bluetooth Speaker	129.99	5.00
Gaming Keyboard	129.99	5.00
Coffee Maker	149.99	5.00
Wireless Earbuds	159.99	4.00
Microwave Oven	179.99	3.00
Smart Watch	199.99	3.50
Noise Cancelling Headphones	249.99	5.00
Ultra HD Monitor	349.99	5.00
Gaming Console Pro	499.99	3.50
Tablet Pro	699.99	5.00
SmartPhone X	899.99	4.50
UltraBook Pro	1299.99	4.50
Desktop Computer	1599.99	4.00

Execution Logs:

Shuchi Tiwari_TechMart_Assign...

SQLAdditions

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	17:44:46	CREATE DATABASE TechMart	1 row(s) affected	0.032 sec
2	17:44:50	USE TechMart	0 row(s) affected	0.000 sec
3	17:44:58	CREATE TABLE Customers (customer_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, last_na...	0 row(s) affected	0.062 sec
4	17:45:05	CREATE TABLE ProductCategories (category_id INT PRIMARY KEY, category_name VARCHAR(50) NOT N...	0 row(s) affected	0.094 sec
5	17:45:13	CREATE TABLE Products (product_id INT PRIMARY KEY, product_name VARCHAR(100) NOT NULL, desc...	0 row(s) affected	0.078 sec
6	17:45:22	CREATE TABLE Orders (order_id INT PRIMARY KEY, customer_id INT, order_date DATETIME DEFAULT C...	0 row(s) affected	0.078 sec
7	17:45:30	CREATE TABLE OrderItems (order_item_id INT PRIMARY KEY, order_id INT, product_id INT, quantity INT N...	0 row(s) affected	0.078 sec
8	17:45:38	CREATE TABLE Employees (employee_id INT PRIMARY KEY, first_name VARCHAR(50) NOT NULL, last_n...	0 row(s) affected	0.078 sec
9	17:45:45	CREATE TABLE Reviews (review_id INT PRIMARY KEY, product_id INT, customer_id INT, rating INT CHEC...	0 row(s) affected	0.094 sec
10	17:45:54	CREATE TABLE Promotions (promotion_id INT PRIMARY KEY, promotion_name VARCHAR(100) NOT NULL...	0 row(s) affected	0.063 sec
11	17:46:01	CREATE TABLE ProductPromotions (product_id INT, promotion_id INT, PRIMARY KEY (product_id, promotio...	0 row(s) affected	0.109 sec
12	17:46:11	INSERT INTO Customers (customer_id, first_name, last_name, email, phone, address, city, state, zip_code, reg...	12 row(s) affected Records: 12 Duplicates: 0 Warnings: 0	0.015 sec
13	17:46:18	INSERT INTO ProductCategories (category_id, category_name, description, parent_category_id) VALUES (1, '...	9 row(s) affected Records: 9 Duplicates: 0 Warnings: 0	0.000 sec
14	17:46:28	INSERT INTO Products (product_id, product_name, description, category_id, price, stock_quantity, manufactu...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0	0.000 sec
15	17:47:02	INSERT INTO Orders (order_id, customer_id, order_date, total_amount, shipping_address, shipping_city, shippi...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0	0.016 sec
16	17:47:12	INSERT INTO OrderItems (order_item_id, order_id, product_id, quantity, unit_price, discount) VALUES (2001, ...	42 row(s) affected Records: 42 Duplicates: 0 Warnings: 0	0.032 sec
17	17:47:21	INSERT INTO Employees (employee_id, first_name, last_name, email, phone, hire_date, job_title, department, ...	15 row(s) affected Records: 15 Duplicates: 0 Warnings: 0	0.016 sec
18	17:47:29	INSERT INTO Reviews (review_id, product_id, customer_id, rating, comment, review_date) VALUES (3001, 1...	20 row(s) affected Records: 20 Duplicates: 0 Warnings: 0	0.016 sec
19	17:47:37	INSERT INTO Promotions (promotion_id, promotion_name, description, discount_percentage, start_date, end_...	7 row(s) affected Records: 7 Duplicates: 0 Warnings: 0	0.016 sec
20	17:47:46	INSERT INTO ProductPromotions (product_id, promotion_id) VALUES (101, 402), -- UltraBook Pro in Back to ...	23 row(s) affected Records: 23 Duplicates: 0 Warnings: 0	0.031 sec
21	17:48:01	SELECT product_id, product_name, price, stock_quantity FROM Products WHERE price > 500 AND stock_q...	3 row(s) returned	0.000 sec / 0.000 sec
22	17:48:12	SELECT CONCAT(first_name, ' ', last_name) AS full_name, email, city FROM Customers WHERE YEAR(registr...	5 row(s) returned	0.000 sec / 0.000 sec
23	17:48:20	UPDATE Customers SET loyalty_points = loyalty_points + 50 WHERE state = 'TX'	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses...	0.000 sec
24	17:50:02	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
25	17:50:16	UPDATE Customers SET loyalty_points = loyalty_points + 50 WHERE state = 'TX'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
26	17:50:16	SELECT customer_id, first_name, last_name, state, loyalty_points FROM Customers WHERE state = 'TX' LIMI...	1 row(s) returned	0.000 sec / 0.000 sec
27	17:50:32	SELECT product_id, product_name, stock_quantity FROM Products WHERE stock_quantity < 30 LIMIT 0, 10...	1 row(s) returned	0.016 sec / 0.000 sec

Shuchi Tiwari_TechMart_Assign...

Limit to 1000 rows

SQLAdditions

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
24	17:50:02	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec
25	17:50:16	UPDATE Customers SET loyalty_points = loyalty_points + 50 WHERE state = 'TX'	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
26	17:50:16	SELECT customer_id, first_name, last_name, state, loyalty_points FROM Customers WHERE state = 'TX' LIMI...	1 row(s) returned	0.000 sec / 0.000 sec
27	17:50:32	SELECT product_id, product_name, stock_quantity FROM Products WHERE stock_quantity < 30 LIMIT 0, 10...	1 row(s) returned	0.016 sec / 0.000 sec
28	17:50:40	SELECT pc.category_name, AVG(p.price) AS avg_price FROM Products p JOIN ProductCategories pc ON p.c...	4 row(s) returned	0.000 sec / 0.000 sec
29	17:50:46	SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS full_name FROM Customers c LEFT JOIN ...	0 row(s) returned	0.000 sec / 0.000 sec
30	17:50:53	SELECT CONCAT(c.first_name, ' ', c.last_name) AS full_name, SUM(o.total_amount) AS total_spent FROM Cu...	5 row(s) returned	0.016 sec / 0.000 sec
31	17:51:02	SELECT p.product_name, p.price, ROUND(AVG(r.rating), 2) AS avg_rating FROM Products p LEFT JOIN Revi...	20 row(s) returned	0.000 sec / 0.000 sec
32	17:51:08	SELECT DISTINCT p.product_name, CONCAT(c.first_name, ' ', c.last_name) AS customer_name FROM Order...	4 row(s) returned	0.000 sec / 0.000 sec
33	17:51:14	SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS employee_name, CONCAT(m.first_na...	15 row(s) returned	0.016 sec / 0.000 sec
34	17:51:22	SELECT p.product_id, p.product_name FROM Products p LEFT JOIN Reviews r ON p.product_id = r.product_...	6 row(s) returned	0.000 sec / 0.000 sec
35	17:51:30	SELECT o.customer_id, CONCAT(c.first_name, ' ', c.last_name) AS customer_name FROM Orders o JOIN Ord...	10 row(s) returned	0.000 sec / 0.000 sec
36	17:51:36	SELECT p.product_id, p.product_name FROM Products p LEFT JOIN ProductPromotions pp ON p.product_id ...	19 row(s) returned	0.000 sec / 0.000 sec
37	17:51:42	SELECT o.customer_id, p.product_name FROM Orders o JOIN OrderItems oi ON o.order_id = oi.order_id JOIN ...	0 row(s) returned	0.016 sec / 0.000 sec
38	17:51:48	SELECT department, COUNT(*) AS num_employees FROM Employees GROUP BY department HAVING COU...	2 row(s) returned	0.000 sec / 0.000 sec
39	17:53:54	SELECT DATE_FORMAT(order_date, '%Y-%m') AS month, SUM(total_amount) AS revenue FROM Orders WH...	8 row(s) returned	0.000 sec / 0.000 sec
40	17:54:01	SELECT pc.category_name, COUNT(*) AS total_orders FROM OrderItems oi JOIN Products p ON oi.product_i...	1 row(s) returned	0.000 sec / 0.000 sec
41	17:54:08	SELECT e.employee_id, CONCAT(e.first_name, ' ', e.last_name) AS name, e.department, e.salary FROM Empl...	5 row(s) returned	0.000 sec / 0.000 sec
42	17:54:23	SELECT p.product_name, COUNT(CASE WHEN r.rating >= 4 THEN 1 END) * 100.0 / COUNT(*) AS perc...	6 row(s) returned	0.016 sec / 0.000 sec
43	17:54:33	SELECT payment_method, ROUND(AVG(total_amount), 2) AS avg_order_value, COUNT(*) AS order...	4 row(s) returned	0.000 sec / 0.000 sec
44	17:54:40	SELECT DAYNAME(order_date) AS day_of_week, COUNT(*) AS order_count FROM Orders GROUP BY day_...	1 row(s) returned	0.000 sec / 0.000 sec
45	17:54:47	SELECT CONCAT(YEAR(o.order_date), 'Q', QUARTER(o.order_date)) AS quarter, pc.category_name, ...	16 row(s) returned	0.000 sec / 0.000 sec
46	17:54:59	CREATE VIEW CustomerPurchaseSummary AS SELECT c.customer_id, CONCAT(c.first_name, ' ', c.last...	0 row(s) affected	0.031 sec
47	17:55:04	SELECT * FROM CustomerPurchaseSummary LIMIT 0, 1000	12 row(s) returned	0.000 sec / 0.000 sec
48	17:55:17	WITH RECURSIVE EmployeeHierarchy AS (SELECT employee_id, manager_id, 1 AS level FROM Employe...	15 row(s) returned	0.015 sec / 0.000 sec
49	17:55:28	SELECT p.product_name, p.price, ROUND(AVG(r.rating), 2) AS avg_rating FROM Products p JOIN R...	14 row(s) returned	0.016 sec / 0.000 sec